TUITIONSHIELD AI

Complete Technical Documentation

2026 Shenzhen International FinTech Competition — International Track

Topic C: Cross-border & Student Finance Assistant

Document Version: 1.0

Document Date: October 2026

Project Name: TuitionShield AI

Project Category: AI-Powered Student Financial Planning and Assistance

Implementation Approach: AI-assisted development, deterministic financial calculations, controlled AI tool calling, and simulated financial operations.

Project Status: MVP developed; implementation and security claims require supporting evidence before final submission.

---

1. EXECUTIVE SUMMARY

TuitionShield AI is an AI-powered financial co-pilot designed to help international students understand, organize, and manage the financial responsibilities associated with studying abroad.

International students must coordinate tuition fees, accommodation, living expenses, scholarships, family support, savings, payment deadlines, and expenses in different currencies. These financial responsibilities are often distributed across different documents, spreadsheets, websites, and conversations. Students may therefore struggle to understand their complete financial situation, identify future funding shortages, or prepare for unexpected changes.

TuitionShield AI addresses this problem through a unified financial planning environment that combines structured financial data, deterministic calculations, scenario analysis, AI-assisted interaction, and controlled simulated payment workflows.

The system is designed to help students answer important questions:

1. How much money will I need to study abroad?
2. How much funding do I currently have available?
3. Which scholarships and other funding sources are confirmed?

4. What expenses and payment deadlines are approaching?
5. Will my current funding cover my expected expenses?
6. What happens if tuition, rent, or exchange rates change?
7. What actions could help reduce a potential funding shortage?
8. How can a payment proposal be reviewed and simulated safely?

The main technical principle is that artificial intelligence assists the student but does not independently control the financial system. Financial totals, funding-gap calculations, authorization decisions, and payment-state transitions must be handled and validated by trusted application logic.

The current MVP uses a web application, Supabase for authentication and PostgreSQL data storage, and the DeepSeek API for AI-assisted functionality. Simulated financial operations are intended to take place in a sandbox environment rather than through real banking infrastructure.

TuitionShield AI is a prototype. It is not a bank, payment institution, lender, investment adviser, or replacement for official financial advice. It does not claim to execute real international transfers.

2. PROBLEM STATEMENT

2.1 Background

Studying abroad involves more than choosing a university and paying tuition. Students must prepare a financial plan that considers the complete cost of education and living in another country.

These costs may include tuition, accommodation, food, transportation, insurance, educational materials, visa-related expenses, and personal spending.

At the same time, students may depend on several sources of funding, including personal savings, family contributions, scholarships, grants, and part-time income.

These sources do not always have the same level of certainty. Savings may already be available, while a scholarship may still be pending or family support may arrive later than expected.

Students also face changing costs and exchange rates. A financial plan that appears sufficient at the beginning of an academic year may become insufficient after an unexpected expense or an increase in living costs.

2.2 Existing Challenges

The principal challenges addressed by TuitionShield AI are:

Financial fragmentation. Students may keep financial information in multiple spreadsheets, documents, applications, and messages.

Limited visibility. Students may know their tuition amount without understanding their complete expected expenses and available funding.

Uncertain funding. Expected scholarships and other unconfirmed funding sources may be incorrectly treated as guaranteed money.

Late risk identification. Students may discover a funding shortage only when a payment deadline approaches.

Scenario uncertainty. Students may find it difficult to calculate the effect of increased rent, delayed scholarship payments, or exchange-rate changes.

Complex financial information. Students may need help understanding their financial data and translating it into practical next steps.

Lack of controlled action workflows. A financial assistant should not merely provide recommendations; where appropriate, it should help users prepare actions while maintaining user control and appropriate safeguards.

2.3 Problem Definition

International students need an accessible and understandable way to organize their financial obligations, distinguish confirmed funding from uncertain funding, estimate future financial shortages, and evaluate possible responses before deadlines are missed.

TuitionShield AI aims to provide this capability through a student-focused financial planning system with a controlled AI assistance layer.

3. PROJECT OBJECTIVES

The main objectives of TuitionShield AI are:

1. Provide a unified overview of student financial information.
2. Organize available funds, funding sources, expenses, obligations, and payment deadlines.
3. Calculate financial totals using deterministic application logic.
4. Identify potential funding shortages and approaching obligations.

5. Support hypothetical financial scenarios without unintentionally changing the original financial plan.
6. Use AI to explain financial information and assist with approved operations.
7. Prepare simulated payment proposals for review and explicit user approval.
8. Maintain appropriate user-data isolation and access control.
9. Record relevant system actions for traceability.
10. Provide an understandable demonstration of AI-assisted financial workflows without moving real money.

The MVP focuses on demonstrating the core workflow. Full banking integration, institution-wide deployment, advanced financial forecasting, and regulatory certification are outside the current verified scope.

4. TARGET USERS AND USE CASES

4.1 Primary Users

The primary users are international students preparing for or currently undertaking education abroad.

Potential secondary users include parents and guardians helping students prepare financial plans, and educational institutions seeking to improve student financial support.

These secondary use cases require further user research and validation.

4.2 Core Use Cases

Use Case 1: Financial Overview

A student enters their available funds, expected expenses, funding sources, and payment obligations. The application organizes the information into a financial overview.

Use Case 2: Funding-Gap Calculation

The student reviews total eligible expenses and available funding. The system calculates whether the current plan indicates a surplus or a shortage.

Use Case 3: Funding Source Management

The student records scholarships, family support, savings, and other funding sources. Each source should have an appropriate status, such as confirmed, pending, or estimated.

Use Case 4: Financial Scenario Analysis

The student evaluates hypothetical changes, such as increased accommodation costs or delayed scholarship payments. The system calculates the effect on the projected financial situation.

Use Case 5: AI-Assisted Explanation

The student asks a question about their financial plan. The AI uses permitted application information to provide an understandable explanation, subject to available data and access permissions.

Use Case 6: Simulated Payment Proposal

The student prepares a simulated payment proposal, reviews its details, and explicitly approves it before any permitted sandbox execution.

Use Case 7: Financial History and Traceability

The student reviews relevant transactions, scenario results, and payment proposal statuses, where these features are implemented.

5. SYSTEM OVERVIEW

TuitionShield AI follows a layered architecture separating the user interface, application services, financial calculations, database, and AI provider.

5.1 User Interface Layer

The user interface is built with React, TypeScript, Vite, and Tailwind CSS.

Its purpose is to provide an accessible environment in which students can enter financial information, review their financial position, explore scenarios, communicate with the AI assistant, and review payment proposals.

The interface should communicate financial information clearly. It should distinguish confirmed amounts from estimates, identify assumptions behind calculations, and visibly label simulated operations.

5.2 Application and Backend Layer

The application layer coordinates requests between the user interface, database, financial calculation modules, and AI provider.

Its responsibilities include input validation, authentication checks, business-rule enforcement, controlled AI access, and the processing of permitted financial operations.

Sensitive operations must not rely exclusively on client-side validation. The trusted server-side layer must validate permissions and relevant business rules before changing financial records or executing sandbox operations.

The current project uses Vite middleware for the DeepSeek integration, according to the development status reported by the team. The deployed configuration must be inspected to confirm that credentials and privileged operations remain protected from browser access.

5.3 Financial Logic Layer

The financial logic layer is responsible for calculations that must produce consistent and testable results.

These calculations include expense totals, available funding, projected funding gaps, scenario differences, and validation of payment proposals.

The language model must not be the authoritative source of financial arithmetic or payment authorization.

5.4 Database Layer

Supabase PostgreSQL stores structured application data. Supabase Auth provides authentication, while PostgreSQL Row-Level Security is intended to restrict access to records according to the authenticated user and applicable permissions.

The team reports that the database is live, nine application tables have been created and seeded, and RLS has been enabled on those tables. These are implementation reports and must be supported by database configuration evidence and access-control tests.

5.5 AI Layer

The DeepSeek API provides the language-model capabilities used by the application.

The AI layer is intended to support conversational assistance, explanation of financial information, and controlled requests to approved application tools.

AI-generated output is treated as untrusted input. The backend must validate tool names, parameters, user permissions, and resulting operations before performing any action.

6. SYSTEM ARCHITECTURE AND DATA FLOW

6.1 High-Level Architecture

The system consists of the following components:

1. Student-facing web application.

2. Authenticated application services and backend middleware.
3. Deterministic financial calculation modules.
4. Supabase authentication and PostgreSQL database.
5. DeepSeek AI API.
6. Sandbox payment workflow and simulated ledger.
7. Automated tests and audit records.

6.2 Normal Financial Request Flow

First, the student interacts with the web application.

Second, the application validates the basic structure of the input and submits the request through the appropriate application service.

Third, the backend establishes the authenticated user context and checks access permissions.

Fourth, the backend retrieves the permitted financial records from Supabase.

Fifth, the financial calculation modules process relevant numerical data using deterministic logic.

Sixth, when an AI explanation is needed, the system provides the AI model with only the information necessary for that task.

Seventh, the application validates the resulting output and displays the result with appropriate explanations and assumptions.

Finally, relevant operations may be recorded in the audit system.

6.3 AI Tool-Calling Flow

When a user requests an action through the AI assistant, the model may propose using an approved application tool.

The backend must verify that the tool is allowed, its arguments meet the required schema, the authenticated user has permission, and the requested operation is consistent with the current application state.

If validation fails, the operation must be rejected safely.

If the operation is permitted, the backend executes it using trusted application logic and returns the result to the AI layer or user interface.

The AI model cannot directly modify database permissions, issue arbitrary SQL, bypass authentication, or authorize its own financial actions.

6.4 Payment Workflow

A simulated payment begins as a proposal rather than an executed transaction.

The application validates the proposal, presents its details to the user, and requests explicit approval.

The system should bind approval to the exact relevant proposal details. A material change to the amount, currency, recipient, or other protected field must invalidate the previous approval.

Only after successful server-side validation of the approval and proposal state may the application perform a sandbox ledger operation.

The user interface must display success only after the backend confirms that the simulated operation was committed successfully.

7. DATABASE DESIGN

The team reports that the MVP contains nine principal database tables.

7.1 Profiles

Purpose: Stores user profile information associated with an authenticated account.

Expected information may include the profile identifier, user preferences, and relevant profile metadata.

Security requirement: A user must not be able to read or modify another user's profile without explicit authorization.

7.2 Financial Accounts

Purpose: Organizes financial account information used by the planning system.

Depending on the actual implementation, this may include account labels, currencies, and reported balances.

Security requirement: Account values must be associated with the correct user. The application must distinguish user-reported information from externally verified financial balances.

TuitionShield AI does not claim to connect directly to real bank accounts in the current MVP.

7.3 Funding Sources

Purpose: Records the sources expected to finance the student's education and living costs.

Examples include savings, family contributions, scholarships, grants, and other income.

Funding sources should distinguish confirmed amounts from pending or estimated amounts. Pending scholarships must not automatically be treated as money already available.

7.4 Financial Obligations

Purpose: Stores expenses and financial obligations that the student expects to pay.

Examples include tuition, accommodation, insurance, transportation, and other living costs.

Where applicable, records should include the amount, currency, due date, category, and payment status.

Security requirement: Obligations must be linked to their owner and validated before being included in calculations.

7.5 Financial Transactions

Purpose: Records financial transaction entries used by the application.

In the current prototype, transaction records must be clearly distinguished from real banking transactions. Sandbox ledger entries represent simulated operations and must never be presented as proof that an external payment occurred.

7.6 Payment Proposals

Purpose: Stores proposed simulated payments and their workflow status.

Relevant information may include the amount, currency, recipient description, linked obligation, approval state, and execution result.

Security requirement: The backend must validate the proposal's owner, current state, and approval status before execution.

7.7 Scenario Runs

Purpose: Stores results from hypothetical financial scenarios.

Examples include increased living costs, delayed scholarship funding, or currency-related changes.

A scenario should preserve the original financial plan unless the user explicitly chooses to modify the baseline through a separate operation.

7.8 Audit Events

Purpose: Records relevant security-sensitive actions and system events.

Examples include payment proposal creation, approval, execution attempts, rejected operations, and important state changes.

Audit records should be protected against unauthorized modification. They should not unnecessarily contain API keys, passwords, authentication tokens, or sensitive conversational content.

7.9 AI Messages

Purpose: Stores AI conversation records where conversation persistence is enabled.

These records may contain personal financial information and should be protected accordingly.

The application should minimize retained information and provide appropriate controls over access and retention.

7.10 Database Integrity Requirements

Every table must be reviewed for correct ownership relationships, appropriate constraints, safe handling of inserts and updates, and appropriate deletion behavior.

Enabling RLS alone does not prove that a table is secure. Each policy must be checked against the intended permissions.

Database access should follow the principle of least privilege. Privileged credentials must never be exposed to the browser or committed to public source control.

8. CORE FINANCIAL ALGORITHMS

8.1 Design Principle

Financial calculations must be deterministic, reproducible, and independent of free-form AI reasoning.

Given the same valid inputs, currency assumptions, and calculation rules, the application should produce the same result.

The system must make its assumptions clear and avoid counting the same financial amount more than once.

8.2 Total Expenses

The total expenses calculation combines the eligible expenses included in a specified planning period.

These may include tuition, living expenses, accommodation, insurance, and other applicable costs.

Only expenses within the selected period and scope should be included. The calculation must avoid double-counting obligations or including already-paid amounts in a measure intended to represent unpaid future expenses.

8.3 Available Funding

Available funding represents the amount of funding eligible to cover the selected obligations under the application's defined rules.

The system should distinguish between confirmed funding already available, confirmed funding expected later, and unconfirmed or estimated funding.

These categories should not be combined without explanation.

The exact treatment of future-dated funds must be defined in the implementation. For example, money expected after a tuition deadline cannot automatically be treated as available before that deadline.

8.4 Funding Gap

The projected funding gap is the difference between eligible expenses and eligible funding.

If eligible expenses exceed eligible funding, the difference represents a projected shortfall.

If eligible funding equals or exceeds eligible expenses, the plan may have sufficient funding for the selected scope, subject to the assumptions and timing of the included amounts.

The application should display whether the result is a projected shortage, a projected surplus, or a balanced plan.

8.5 Illustrative Calculation

Consider the following example:

Available funds: €8,000.

Tuition expenses: €4,000.

Living expenses: €5,500.

Other expenses: €1,000.

Total listed expenses: €10,500.

Projected funding gap: €2,500.

This example assumes that all listed expenses are eligible costs for the same planning period, that the €8,000 represents eligible available funding, and that no expense or funding amount has been counted twice.

The example is illustrative and is not a claim about an actual student's finances.

8.6 Currency Handling

The system may need to organize obligations and funding in different currencies.

Before combining amounts in different currencies, the application must convert them into a selected reporting currency using an explicitly identified exchange rate.

The calculation should retain the source currency, exchange-rate assumption, and conversion time or date when available.

If no reliable exchange rate is available, the system must not silently invent one. It should request an input or clearly identify that the calculation cannot be completed accurately.

The current MVP should not claim to provide live exchange rates unless a verified exchange-rate data source has actually been integrated.

8.7 Scenario Analysis

Scenario analysis evaluates the effect of hypothetical changes to the financial plan.

Examples include an increase in monthly rent, a delayed scholarship, an unexpected expense, or an exchange-rate movement.

The application should calculate the resulting financial difference using the same deterministic calculation rules used for the baseline plan.

Scenario results must be distinguishable from actual financial records. Running a scenario should not unintentionally change the student's original funding sources, obligations, or transaction history.

8.8 Financial Data Validation

The application should reject invalid or inconsistent financial inputs.

Examples include negative values where they are not permitted, unsupported currencies, malformed dates, missing required fields, and amounts outside defined limits.

Monetary values should use a representation that avoids inappropriate floating-point rounding. Calculations should follow consistent currency precision and rounding rules.

8.9 Algorithm Limitations

Financial forecasts depend on the completeness and accuracy of user-provided data.

The system cannot guarantee that scholarships will be awarded, future costs will remain stable, or all expenses have been recorded.

All projections should therefore be presented as estimates based on explicit assumptions rather than guarantees.

9. AI AGENT DESIGN

9.1 Purpose

The AI agent is designed to make the financial planning system easier to use by interpreting natural-language requests, explaining calculations, summarizing available financial information, and requesting approved operations through controlled application tools.

The agent is not the authoritative financial calculation engine and does not independently control payment execution.

9.2 DeepSeek Integration

The application uses the DeepSeek API for AI-assisted functionality.

The backend should provide the model with the minimum context needed to answer the current request. Sensitive information should not be transmitted unnecessarily.

API credentials must be stored in an appropriate server-side secret configuration. They must not be exposed through client-side environment variables or included in publicly accessible source code.

The current team-reported integration uses Vite middleware. Its deployment and production security configuration must be reviewed before the project claims that all AI requests are securely handled server-side.

9.3 Approved Tool Categories

The agent may be designed to request a limited set of approved operations, such as:

1. Retrieving the authenticated user's permitted financial summary.
2. Calculating a funding gap.
3. Retrieving relevant obligations and due dates.
4. Running a hypothetical scenario.
5. Creating a simulated payment proposal.

6. Retrieving the status of an existing proposal.

Only tools actually implemented and validated should be described as available in the submitted MVP.

9.4 Tool Validation

Every tool request must be validated by the application.

Validation should confirm that the requested tool exists, its arguments are valid, the authenticated user is authorized, and the requested operation is consistent with the current application state.

The application must not trust a model-generated user identifier, payment status, balance, approval result, or transaction identifier merely because the model supplied it.

The backend must derive identity and permissions from trusted authentication context.

9.5 Handling AI Errors

AI models may misunderstand a request, generate incorrect explanations, or propose unsuitable actions.

The application should handle these cases by rejecting invalid tool calls, requesting clarification where necessary, and explaining when reliable information is unavailable.

Financial calculations must be performed by trusted application logic and then supplied to the model for explanation where appropriate.

The model must not be allowed to invent missing financial values and present them as verified facts.

9.6 AI Prompt Injection and Untrusted Content

Financial documents, user messages, and retrieved text may contain instructions that attempt to manipulate the AI agent.

Such content must be treated as data rather than as permission to override system rules.

The model must not be able to bypass authentication, reveal protected records, change access policies, or execute unauthorized operations through instructions contained in a document or conversation.

10. SANDBOX PAYMENT WORKFLOW

10.1 Purpose and Scope

The payment workflow demonstrates how an AI-assisted application can help a user prepare and review a financial action while preserving user control.

All payment operations in the current MVP are intended to use simulated data and a sandbox ledger.

No real funds are transferred, and the application does not claim to initiate actual tuition payments through banks or payment providers.

10.2 Payment Proposal Lifecycle

The intended payment proposal lifecycle consists of the following stages:

Draft. The proposal is created with the requested payment details.

Validated. The application checks the proposal's structure, amount, currency, ownership, and applicable business rules.

Awaiting Approval. The proposal is presented to the authenticated user for review.

Approved. The user explicitly approves the exact validated proposal.

Executing. The backend attempts the permitted sandbox operation.

Executed. The backend confirms that the simulated ledger operation has completed successfully.

Rejected. The proposal is declined or fails validation.

Failed. The execution attempt fails without a confirmed successful ledger operation.

Expired. The proposal is no longer valid under the application's defined expiration rules.

These states describe the intended workflow. The team must verify which states and transitions are implemented in the current application.

10.3 Explicit Approval

The application must present the relevant payment details before approval.

The user should be able to review the amount, currency, recipient description, and linked obligation where applicable.

Approval must be a deliberate user action. An AI-generated recommendation or tool call must not count as approval.

The backend must validate the authenticated user's authority and confirm that the approved proposal has not materially changed.

10.4 Duplicate Execution Prevention

Repeated requests must not create duplicate simulated transactions.

The system should use an appropriate idempotency mechanism or equivalent transaction-level protection to ensure that retries do not duplicate an already completed operation.

The backend should also handle concurrent requests and partial failures safely.

The team must test this behavior rather than assume that it is provided automatically by the database.

10.5 Failure Handling

If the database or application fails during execution, the interface must not display a successful payment unless the backend confirms the simulated ledger operation.

Where the outcome is uncertain, the system should check the existing transaction state before retrying.

Errors should be recorded appropriately without exposing secrets or unnecessary sensitive data.

10.6 Payment Security Requirements

The following conditions must be verified:

1. A proposal cannot be executed without valid approval.
2. Approval cannot be reused after a material change to protected proposal details.
3. One user cannot execute another user's proposal.
4. An already executed proposal cannot produce a duplicate ledger entry.
5. Invalid or expired proposals are rejected.
6. Execution failures do not produce false success messages.
7. Relevant state transitions are recorded for review.
8. The user interface clearly identifies every payment as simulated.

11. SECURITY ARCHITECTURE

11.1 Security Objectives

The principal security objectives are confidentiality, integrity, controlled authorization, availability, and traceability.

Confidentiality means that users must not access another user's private financial records.

Integrity means that financial values, transaction entries, approval states, and payment results cannot be altered through unauthorized requests.

Authorization means that sensitive operations are performed only when the applicable permissions and conditions are satisfied.

Traceability means that important actions can be reviewed through appropriate audit records.

11.2 Authentication

Supabase Auth is used for application authentication.

The backend must verify the authenticated identity before permitting access to protected operations.

A user identifier supplied by the browser must not be treated as proof of identity.

Authentication failures should be handled safely, and expired or invalid sessions must not authorize protected operations.

11.3 Row-Level Security

PostgreSQL Row-Level Security is intended to restrict access to records according to the authenticated user's identity and the application's authorization model.

Each of the nine reported tables must be reviewed individually.

The review should include SELECT, INSERT, UPDATE, and DELETE operations where those operations are permitted.

Testing must include attempts to access another user's records, insert records under another user's identity, modify protected records, and delete records without authorization.

The fact that RLS is enabled does not establish that every policy is correct.

11.4 API Key Protection

The DeepSeek API key and any privileged database credentials must remain server-side.

They must not be placed in publicly exposed browser variables, committed to a public repository, included in screenshots, or returned in error messages.

The deployed application must be inspected to confirm that browser requests cannot retrieve these secrets.

If a credential is accidentally exposed, it should be revoked or rotated.

11.5 Input Validation

All important inputs must be validated on the trusted application side.

This includes financial amounts, currencies, dates, user identifiers, payment proposal fields, AI tool arguments, and database operations.

Client-side validation may improve usability but must not replace backend validation.

11.6 Least Privilege

Each component should receive only the permissions needed for its responsibilities.

The AI model should not receive database administrator privileges. The browser should not receive privileged database credentials. Payment execution should be restricted to the appropriate authenticated and approved workflow.

11.7 Audit Protection

Audit events should record relevant actions, outcomes, timestamps, and object references.

Audit records should not expose passwords, API keys, authentication tokens, or unnecessary personal financial information.

Users should not be able to freely rewrite or erase security-sensitive audit events through ordinary application operations.

11.8 Privacy and Data Minimization

The application should collect only information needed for its financial-planning purpose.

Financial information and AI conversations should be treated as potentially sensitive personal data.

The project should clearly disclose the role of the external AI provider and avoid sending unnecessary personal information to it.

Data retention, deletion, and privacy practices must be documented according to the actual implementation.

11.9 Security Limitations

The MVP has not been independently penetration-tested, unless separate evidence establishes otherwise.

No financial-services regulatory certification is claimed.

The application does not claim to be fully secure. Its security status must be evaluated against implemented controls, reproducible tests, and the limitations of the current deployment.

12. SECURITY THREAT MODEL

12.1 Unauthorized Data Access

Threat: A user attempts to retrieve another user's financial information.

Required mitigation: Authentication, database-enforced ownership policies, and cross-user access tests.

12.2 Unauthorized Payment Execution

Threat: A user or AI-generated tool request attempts to execute a proposal without approval.

Required mitigation: Server-side authorization, explicit approval, proposal-state validation, and negative execution tests.

12.3 Approval Tampering

Threat: Proposal details are changed after the user approves them.

Required mitigation: Bind approval to the exact protected proposal details and invalidate approval after a material change.

12.4 Duplicate Operations

Threat: A repeated request causes the same simulated payment to be recorded multiple times.

Required mitigation: Idempotency controls, transaction-safe state transitions, and retry tests.

12.5 AI Hallucination

Threat: The AI invents financial values, incorrectly explains a calculation, or recommends an action based on unsupported assumptions.

Required mitigation: Deterministic calculations, source-aware explanations, input validation, and clear uncertainty labels.

12.6 Prompt Injection

Threat: Malicious instructions inside user content or imported documents attempt to override the agent's permitted behavior.

Required mitigation: Treat retrieved content as untrusted data, restrict tool permissions, and enforce authorization outside the model.

12.7 Credential Exposure

Threat: An API key or privileged database credential becomes accessible through client code, logs, or repository history.

Required mitigation: Server-side secret management, repository checks, careful logging, and credential rotation after exposure.

12.8 Incorrect Financial Inputs

Threat: Incomplete or inaccurate information causes misleading financial projections.

Required mitigation: Input validation, explicit assumptions, user confirmation, and warnings that projections depend on provided data.

13. TESTING AND VALIDATION PLAN

13.1 Purpose

Testing must demonstrate that the application performs its intended functions correctly and rejects unsafe operations.

The team has reported that 28 out of 28 automated tests pass. This result should be supported by actual test output, test descriptions, and reproducible execution instructions.

A passing test count alone does not prove that every important security condition has been tested.

13.2 Financial Calculation Tests

Testing should verify that expense totals are correct, funding-gap calculations use the correct inputs, confirmed and pending funding are treated according to the defined rules, and rounding is consistent.

Tests should include balanced plans, funding shortages, funding surpluses, zero amounts, invalid inputs, and different currency assumptions.

13.3 Scenario Tests

Testing should verify that hypothetical changes produce the expected financial difference and do not unintentionally modify the original plan.

Scenarios should include increased living expenses, delayed funding, and unexpected costs.

13.4 Authentication and Database Tests

Testing should verify that authenticated users can access their permitted records and cannot access another user's protected data.

The team should test each relevant database operation and inspect the applicable RLS policies.

13.5 AI Tests

Testing should cover valid requests, malformed tool arguments, unknown tools, missing information, unsupported financial claims, provider errors, and rejected operations.

The system should remain safe when the AI provider is unavailable or returns an invalid response.

13.6 Payment Workflow Tests

Testing should cover the complete valid sandbox workflow, rejection without approval, rejection after proposal tampering, duplicate execution attempts, expired proposals, and failure handling.

The test evidence should demonstrate the resulting proposal state and ledger behavior.

13.7 Deployment and Secret Tests

Testing should confirm that the application builds and starts correctly, protected routes require appropriate authorization, and sensitive API keys are not exposed to the browser.

Deployment instructions should allow another reviewer to reproduce the relevant setup and tests without access to private credentials.

13.8 Test Evidence

The submission evidence should include actual automated test output, relevant database policy definitions, screenshots or logs showing denied unauthorized actions, and sandbox execution logs.

Only tests actually executed should be marked as passed.

Any failed test should be documented with its impact, mitigation, and release status.

14. SAMPLE DEMONSTRATION SCENARIO

The following scenario is an illustrative demonstration plan. It must not be represented as a completed execution unless the corresponding actions have been performed in the live MVP.

14.1 Initial Financial Information

The demonstration student has €8,000 in eligible available funds.

The student records €4,000 in tuition expenses, €5,500 in living expenses, and €1,000 in other expenses.

The system calculates total listed expenses of €10,500 and a projected funding gap of €2,500.

The interface explains the assumptions behind this result.

14.2 Scenario Analysis

The student increases the expected living expenses in a hypothetical scenario.

The system recalculates the projected funding gap using the changed scenario inputs.

The baseline financial plan remains unchanged unless the user explicitly updates it through a separate supported action.

14.3 AI Assistance

The student asks why the plan shows a funding shortage.

The AI explains the calculated result using the trusted application output and identifies the expenses and funding assumptions that contribute to it.

The AI should not invent scholarship awards, exchange rates, or financial information that is absent from the available data.

14.4 Simulated Payment

The student creates a simulated payment proposal for an eligible obligation.

The application validates the proposal and presents the details for review.

The student explicitly approves the proposal.

The backend checks the approval and proposal state, performs the permitted sandbox operation, and confirms the resulting ledger entry.

The interface displays the result as a simulated transaction, not a real transfer.

14.5 Security Demonstration

A second test attempts to access the first user's protected records.

The system should reject the unauthorized request.

A separate test attempts to execute a proposal without approval.

The system should reject that operation as well.

The team should present the actual resulting test output or logs to demonstrate these protections.

15. IMPLEMENTATION STATUS AND LIMITATIONS

15.1 Team-Reported Implementation

According to the development status reported by the team:

- The web application has been developed as an MVP.
- Supabase is connected and the database is live.
- Nine application tables have been created and seeded.
- RLS has been enabled on the nine reported tables.
- DeepSeek is connected through Vite middleware.
- Twenty-eight automated tests reportedly pass.

These statements are based on team reports and are not independent verification results. The final submission should include evidence supporting each claim.

15.2 Controls Requiring Verification

The team must confirm the actual implementation and test results for:

- Correct RLS policies and cross-user isolation.
- Server-side validation of sensitive operations.
- Protection of API keys and privileged credentials.
- Deterministic financial calculations.
- AI tool restrictions and error handling.
- Explicit approval for simulated payment execution.
- Approval invalidation after proposal changes.
- Duplicate execution prevention.
- Audit-event integrity.
- Safe failure behavior.

A control should not be described as implemented merely because it appears in the architecture or documentation.

15.3 Current Limitations

TuitionShield AI is a prototype intended for demonstration and further validation.

It does not currently claim real bank connectivity or real-money transfers.

Financial forecasts depend on user-provided information and defined assumptions.

AI-generated explanations may be inaccurate and must not override trusted calculations or authorization rules.

External AI processing introduces a third-party data-handling dependency.

The application has not been independently penetration-tested unless separate evidence establishes otherwise.

No claim of regulatory certification or complete financial-services compliance is made.

The current prototype does not guarantee scholarship availability, future financial stability, or the accuracy of every user-entered value.

15.4 Future Development

Potential future improvements include document import with user confirmation, more advanced scenario modeling, verified exchange-rate integration, improved forecasting, and pilot testing with international students.

Real payment integration, if considered in the future, would require a separate security, legal, compliance, and operational assessment. It is not part of the current sandbox-only scope.

16. ETHICAL AND RESPONSIBLE AI PRINCIPLES

TuitionShield AI is designed to assist students in understanding their finances while preserving their control over decisions.

The system should distinguish factual financial inputs from estimates and recommendations.

It should explain the assumptions behind its calculations and communicate uncertainty where appropriate.

It should not fabricate scholarships, guarantee financial outcomes, or present unsupported advice as verified information.

Sensitive financial data should be handled carefully, and the system should minimize unnecessary disclosure to external services.

The AI must not independently authorize financial actions. Any simulated payment operation must follow the application's explicit approval and validation workflow.

These principles are intended to make the system more understandable, auditable, and responsible.

17. REPRODUCIBILITY AND SOURCE CODE

The competition requires source code, a README, a repository link, deployment instructions, and test cases.

The source repository should contain the application code, relevant configuration templates, documentation, and reproducible test instructions.

Secrets, private credentials, production environment files, and sensitive user data must not be committed to the repository.

The README should explain the project purpose, prerequisites, installation process, environment configuration, database setup, development commands, production build, deployment process, and testing instructions.

Database setup instructions should describe how the required schema and policies are applied.

Test instructions should identify the commands used to run the tests and should include the actual results obtained by the team.

Deployment instructions must not expose privileged credentials. Reviewers should be able to understand the setup without receiving private keys or passwords.

The repository and deployment instructions must accurately describe the current implementation rather than an intended future version.

18. COMPETITION DELIVERABLES

The competition documentation identifies six submission components.

First, technical documentation. This document describes the system architecture, core algorithms, database design, AI agent, security controls, payment workflow, and testing approach.

Second, presentation deck. The final presentation must not exceed ten minutes.

Third, demo video. This is optional and must not exceed five minutes.

Fourth, source code. The repository should include a README, repository link, deployment instructions, and test cases.

Fifth, security self-assessment report. This must describe permission-tier implementation and known risks, supported by evidence where possible.

Sixth, execution evidence. For a non-on-chain topic such as Topic C, the competition requests simulated-account or sandbox operation logs and demo video evidence.

The team should verify that the technical documentation, presentation, security report, source repository, and demo all describe the same actual implementation.

19. CONCLUSION

TuitionShield AI proposes a controlled AI-assisted approach to a practical problem faced by international students: organizing education-related finances and identifying potential funding shortages before they become serious difficulties.

The system combines structured financial information, deterministic calculations, hypothetical scenario analysis, AI-assisted explanations, and a simulated payment workflow with explicit user approval.

Its principal technical distinction is the separation between the AI assistant and the trusted financial control layer. The AI helps users understand information and request permitted operations, while application logic remains responsible for financial calculations, authorization, and payment-state validation.

The MVP provides a foundation for demonstrating this approach. Its credibility depends on verifying the implementation, documenting its limitations, and providing reproducible evidence of both successful and rejected operations.

TuitionShield AI aims to make the financial side of international education clearer, more understandable, and easier to plan—while keeping students informed and in control.

APPENDIX A: FINAL VERIFICATION CHECKLIST

Before submitting, the team should confirm each item below against the actual project.

1. The architecture description matches the current application.
2. The nine reported database tables exist in the intended environment.
3. The RLS policies have been reviewed and tested.
4. Cross-user data access is rejected.
5. Financial calculations have reproducible tests.

6. Currency assumptions and rounding behavior are documented.
7. AI tools are restricted and validated by trusted application logic.
8. API keys and privileged credentials remain protected.
9. Payment proposals cannot execute without valid approval.
10. Material changes invalidate previous approval.
11. Duplicate sandbox execution is prevented.
12. Failed operations do not produce false success messages.
13. Audit events contain appropriate information and exclude secrets.
14. The reported automated test results can be reproduced.
15. The README and deployment instructions are complete.
16. The source repository contains no exposed secrets.
17. The security self-assessment identifies unverified controls and known risks.
18. Sandbox logs demonstrate actual successful and rejected operations.
19. The demo clearly distinguishes simulated activity from real financial transactions.
20. The presentation and technical documentation do not claim unimplemented features as completed.

APPENDIX B: DOCUMENTATION DECLARATION

This document describes the intended architecture, technical approach, and team-reported status of TuitionShield AI.

Features described as intended, required, or proposed must not be treated as verified implementation unless supported by the current source code, configuration, and test evidence.

The team should update this document whenever the implementation changes and before final submission.

Project: TuitionShield AI.

Competition: 2026 Shenzhen International FinTech Competition — International Track.

Topic: Cross-border & Student Finance Assistant.